Improving Selectivity Estimation for Filtered Vector Search in an Embedded Vector Database

CS257 Database System Principles, Fall 2026: Milestone 1 Status Report

Team: Aishwarya Ramanath Shanbhag (020742865),
Anushka Siri Raghunandan (020752979),
Keerthana Sudheer (020776639)

Code: https://github.com/keethz/MicroNN

1. Team Organization

The paper evaluates MicroNN on three public million-scale datasets (SIFT, GloVe and GIST), so each of us runs the experiments on one of them. Next, each of us will try one new idea on top of the re-implementation and test whether it improves on the paper.

Member	Role	What is done so far
Keerthana Sudheer	Core system and research extension A	Re-implemented MicroNN in Python on SQLite (storage, clustering, search, filtering, optimizer, batching, updates, transactions), wrote the test suite, ran the SIFT1M experiments, and built the correlation-aware selectivity estimators.
Aishwarya Ramanath Shanbhag	GloVe experiments, research extension B, reports	Built adaptive filtered search (a new search plan that decides between pre- and post-filtering while the query runs), with its tests, experiment scripts and a SIFT-like synthetic benchmark. Set up the GloVe dataset to match the paper, made the benchmark scripts run on Windows, and ran all the paper's experiments and both research extensions on GloVe. Wrote the status report, and drew the architecture diagram.

Member	Role	What is done so far
Anushka Siri Raghunandan	GIST1M experiments, focused benchmark	Added GIST1M loading and a reproducible 100K/200K workflow. Implemented recall-vs-probe and research experiments for estimator q-error and optimizer trade-offs; full 1M runs are out of scope.

2. System Configuration

The whole project runs on our own laptops. MicroNN is meant for a single small device, so we do not need a server or any cloud resources.

Item	Configuration
Software	Python 3.9+ (tested on 3.9.6 and 3.13), NumPy 2.x, matplotlib, psutil, pytest
Database	SQLite 3 from the Python standard library, in WAL mode, with a 2 MB page cache to act like a small device
Hardware (SIFT1M runs)	Apple M5 laptop, 10 cores, 16 GB RAM, macOS
Hardware (Aishwarya)	Dell laptop, 13th Gen Intel Core i7-1355U, 16 GB RAM, 64-bit Windows
Hardware (Anushka)	Apple M5 Pro — 15 cores (5 Super, 10 Performance) — 24 GB memory
Data	SIFT1M (1M vectors, 128 dimensions); GloVe (1.18M vectors, 200 dimensions, ANN-Benchmarks glove-200-angular, matching the paper's GloVe [2, 3]); GIST1M (1M vectors, 960 dimensions; scripts ready for 100K and 200K subsets); synthetic data for development
Code hosting	GitHub repository keethz/MicroNN, shared by all three of us

3. Project Definition

Our project is to re-implement MicroNN [1], a small vector database from Apple that keeps its index on disk inside SQLite, check whether the paper's main claims hold in our version, and then improve how it handles filtered queries.

Functional requirements

1. Store vectors and their attributes in SQLite, with each cluster of vectors stored together on disk.
2. Build an IVF index using mini-batch k-means, with an option to keep cluster sizes balanced.
3. Return the top-k nearest neighbours by scanning only the nprobe closest clusters.
4. Support filtered queries (=, <, >, BETWEEN, IN, AND) using either a pre-filter or a post-filter plan.
5. Let an optimizer choose the plan automatically from selectivity estimates.
6. Answer a batch of queries together, sharing cluster scans.
7. Allow inserts and deletes through a delta store, with merges and full rebuilds.
8. Research: make plan choice work better when filters are correlated.

Non-functional requirements

1. Memory: about 10 MB per query at 1 million vectors, as the paper claims.
2. Speed: a few milliseconds per query at about 90% recall@100.
3. Correctness: the pre-filter plan returns exact results, and probing every cluster gives the same answer as brute force.
4. Consistency: a search sees one snapshot of the data even while a merge runs, and a crash never leaves half-written data.
5. Reproducibility: fixed random seeds, and every experiment is one script that saves its results to files.

4. Architecture

MicroNN is an embedded library, not a server. The application calls Python functions, and the library keeps everything in one SQLite file. Only the cluster centres and one cluster at a time sit in memory, which is how memory stays around 10 MB.

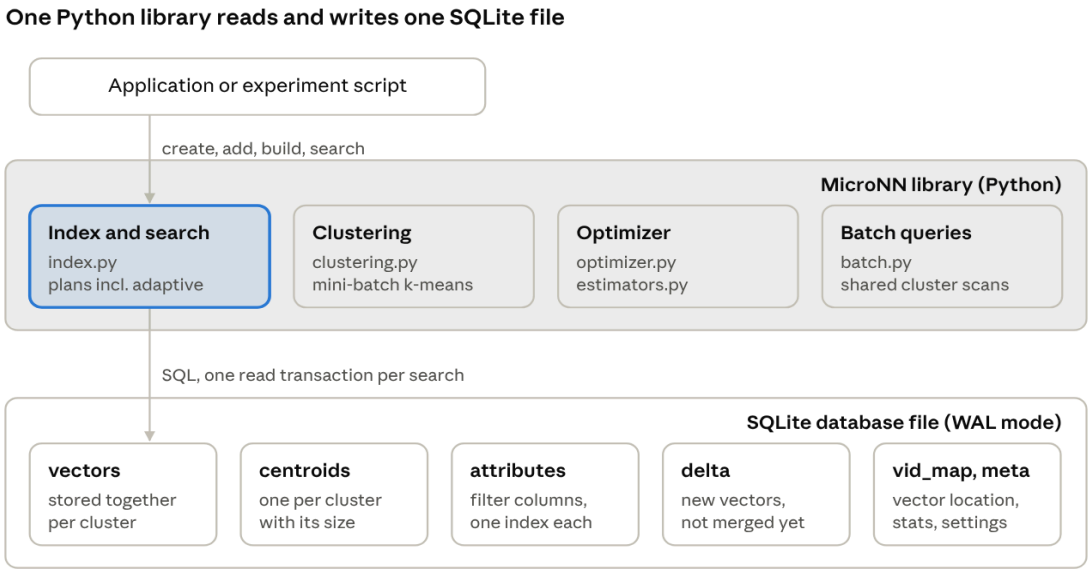


Figure 1. Architecture: four library modules on top of five SQLite tables.

How a filtered query runs:

1. The query vector is compared with every cluster centre in memory, which ranks the clusters.
2. The optimizer picks a plan: pre-filter, post-filter, or our new adaptive plan, which decides while the query runs.
3. The plan reads from SQLite: the attribute index for pre-filter, or the nearest clusters joined with their attributes for post-filter.
4. Rows go into a running top-k list, the delta store is scanned last, and the k nearest results are returned. Everything runs in one read transaction, so a merge happening at the same time cannot change the answer halfway.

5. Scope So Far

The base system and research extension A are finished and tested on SIFT1M and GloVe. Research extension B (adaptive search) is finished and tested on synthetic data and GloVe. The re-implementation is in place, so we are about halfway through: the second half is a study where each of us tries one new idea on top of it.

5.1 Research and evaluation methodology

- We evaluate SIFT, GloVe and GIST, the three public million-scale datasets from the paper's Table 2. MNIST and NYTimes are too small to show disk and memory effects, DEEPImage (10M vectors) exceeds our laptops, and InternalA

is not public. We use the full base set of SIFT and GloVe (GIST will use 100K and 200K subsets) and a random subset of the paper's queries (30 to 2,048 per experiment instead of 10,000) to keep run times practical.

- Recall means recall@100 against exact search over the same vectors and the same filter. Latency is the median (and p95) over 30 to 2048 queries, measured after a warm-up.
- The filter attributes are synthetic but attached to the real SIFT and GloVe vectors: uniform, skewed (Zipf), correlated pairs of columns, and columns tied to where the vector sits in space.
- Python is a lot slower than the paper's native code, so we compare trends and ratios rather than exact milliseconds.

5.2 Implementation details

- **Storage:** vectors live in a WITHOUT ROWID table keyed on (partition_id, vid), so each cluster is stored in one place on disk. Other tables hold the centroids, the attributes (one index per column), the delta store, and a map from vector id to cluster.
- **Search:** we find the nprobe nearest centroids in memory, read those clusters one at a time into a running top-k, and then scan the delta store.
- **Filtering:** the pre-filter plan uses the attribute index and computes exact distances. The post-filter plan scans the chosen clusters joined with the attributes.
- **Optimizer:** statistics per column, combined assuming the columns are independent (the paper also assumes independence, but takes the minimum over a conjunction where our code multiplies; we will check this difference), plus a cost model measured on the machine and a recall guard.
- **Research extension A:** three estimators that handle correlated filters: joint statistics for column pairs, a uniform sample, and a sample spread evenly over clusters.
- **Research extension B (beyond the proposal):** a new `plan="adaptive"`. Instead of guessing the plan up front, it scans clusters nearest-first with the filter on, measures how many rows match near the query (s), keeps going to about nprobe / s clusters, and switches to the exact pre-filter if that would cost more.
- **Testing:** 51 unit tests, including checks that full search equals brute force, pre-filter is exact, batched results equal single results, a crash during a merge loses nothing, and readers never see duplicates.

- **Portability:** the benchmark scripts used a Mac/Linux-only module to measure memory; we changed this so they also run on Windows. We also made the scripts save result files as UTF-8, since Windows' default encoding crashed on the '≥' symbol.

5.3 Requirements done so far

Requirement	**Status**
F1 to F7 (storage, index, search, filters, optimizer, batching, updates)	Done and tested
F8 (research on correlated filters)	Extension A done on SIFT1M and GloVe; extension B done on synthetic data and GloVe
N1 to N4 (memory, speed, correctness, consistency)	Measured on SIFT1M and GloVe, see below
N5 (reproducibility)	Done: one script per experiment, results saved as JSON and plots
Datasets from the paper	SIFT1M done; GloVe done; GIST1M in progress (loader and scripts ready, runs not done yet)

5.4 Preliminary results: the paper's claims on SIFT1M

#	**Paper claim**	**Our result**	**Verdict**
C1	Under 7 ms at 90% recall	89% recall at 7.1 ms; 96% at 14.5 ms	Holds roughly
C2	About 10 MB, about 100x less than in-memory	10.8 MB vs 655 MB (61 to 86x less)	Holds
C3	Disk speed close to in-memory	About 7.5x slower, mostly Python driver overhead	Does not hold in Python
C4	Bounded build memory, balanced clusters	Build peak 94 MiB; cluster size std 491 down to 101	Holds
C5	Pre-filter exact; plans cross over	Pre-filter recall 1.00 everywhere; crossover near 1%	Holds

#	Paper claim	Our result	Verdict
C6	Optimizer picks the right plan	Within ~10% of the fastest plan, but recall only 0.73 to 0.90 for 1 to 10% filters	Partly holds
C7	Batching saves about 33%	92 to 95% faster per query at batch 1024	Holds, much stronger
C8	Updates stay fast and correct	10 to 28 ms with merges; 237 ms if never merged	Holds if merged
C9	Transactions and snapshots for free	Works, but only after we fixed a snapshot bug	Holds, not for free

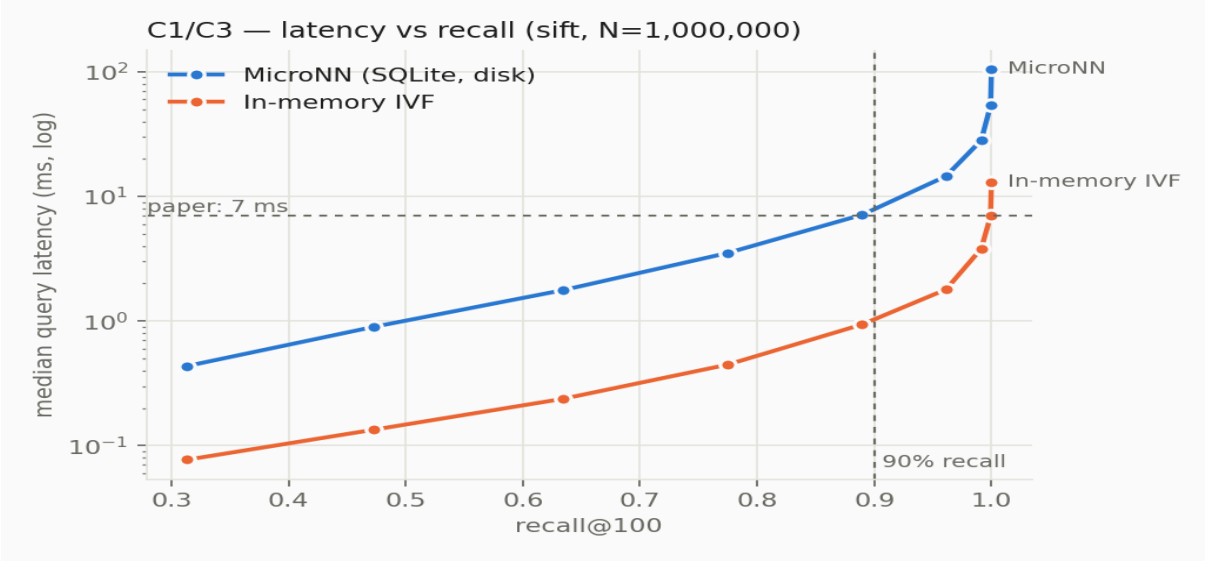


Figure 2. Latency vs recall@100 on SIFT1M: MicroNN on disk vs an in-memory index.

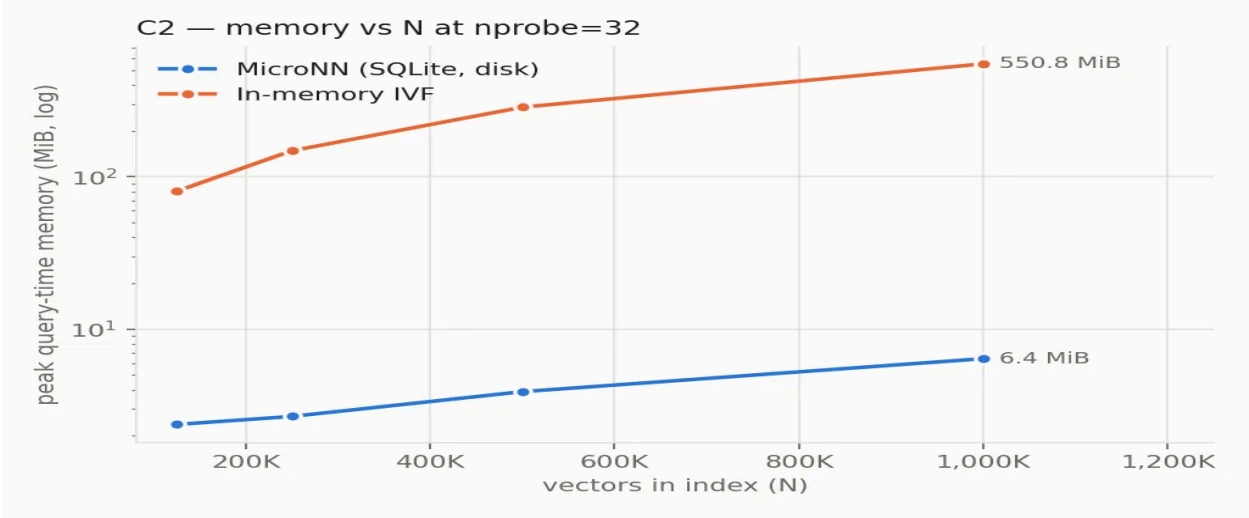


Figure 3. Peak query memory vs dataset size on SIFT1M: MicroNN stays at a few MB, while the in-memory index grows past 550 MB.

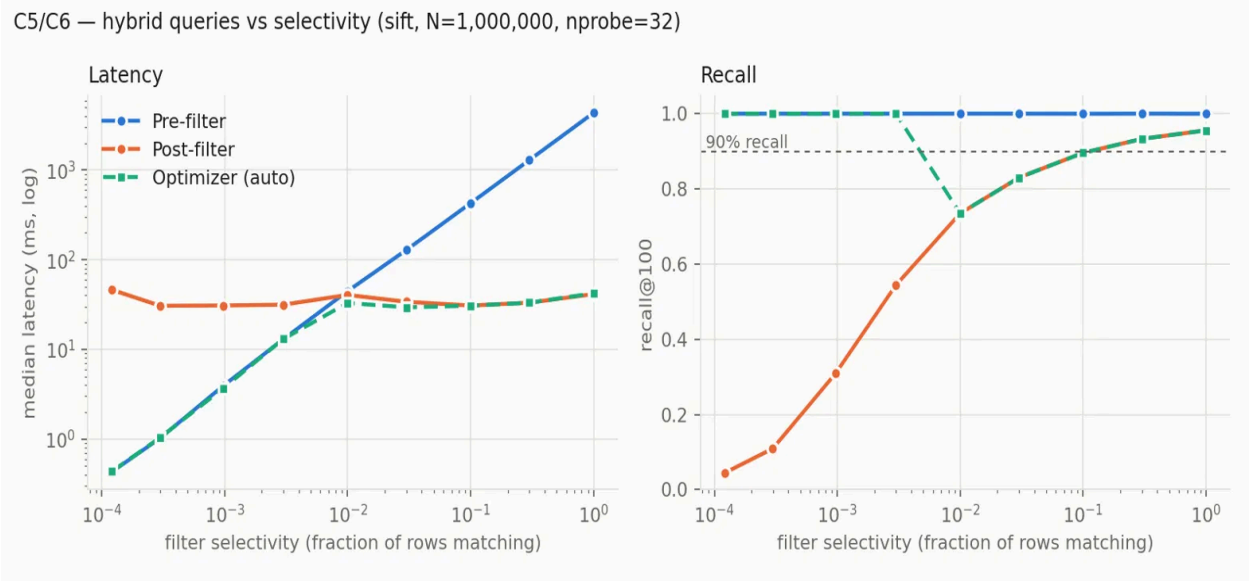


Figure 4. Filtered queries on SIFT1M: pre-filter is always exact; post-filter is faster for broad filters but loses recall below about 1% selectivity.

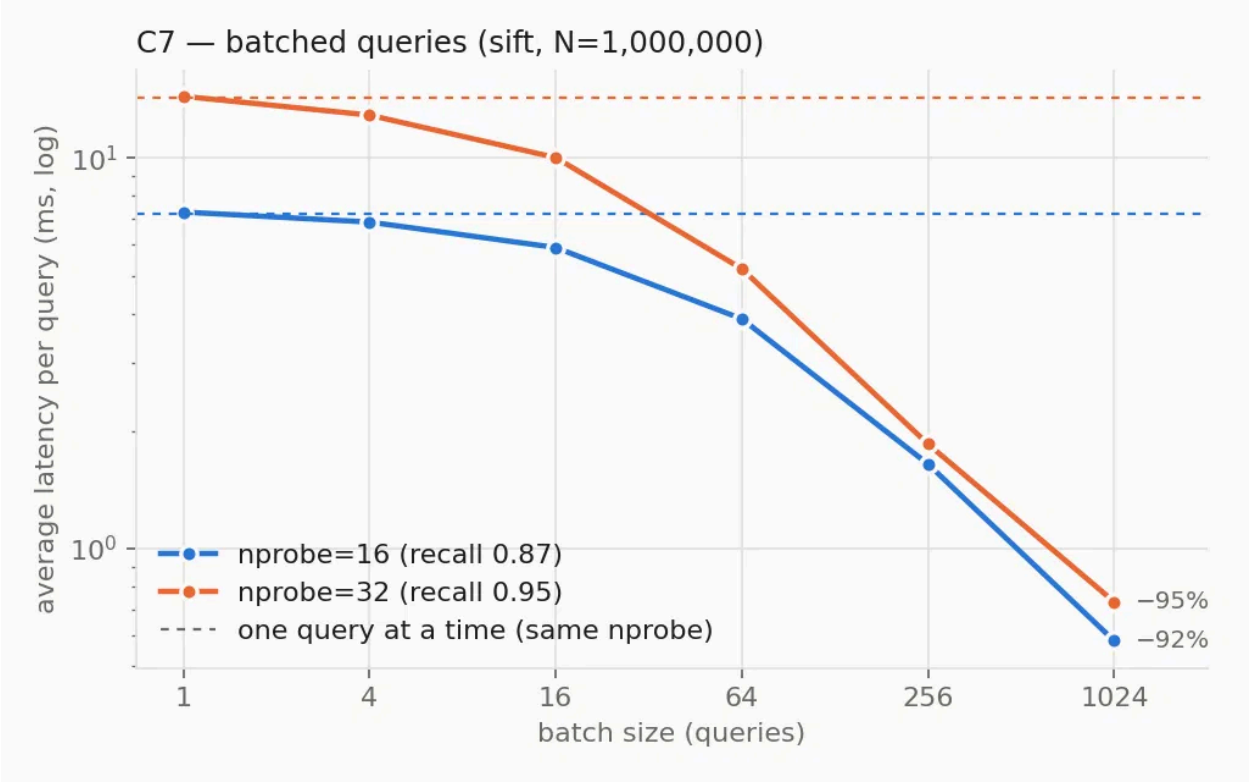


Figure 5. Batching on SIFT1M: per-query latency drops by 92 to 95% at batch size 1024.

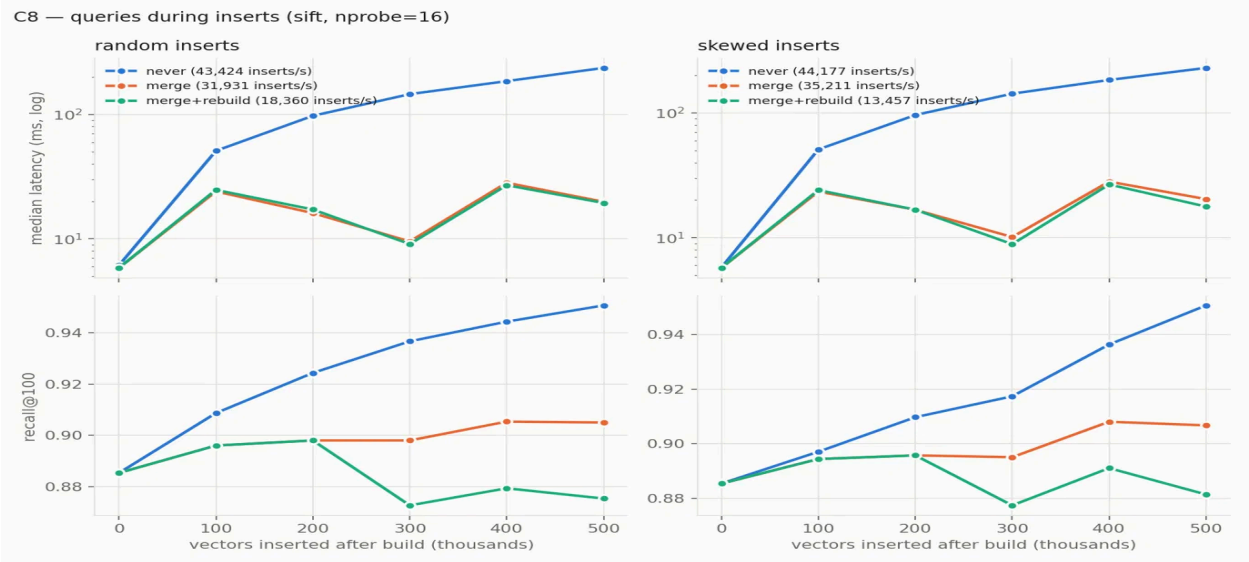


Figure 6. Queries during 500K inserts: without merging, latency keeps growing; merging keeps it at 10 to 28 ms.

5.5 Preliminary results: extension A (SIFT1M)

- The paper's independence assumption gets correlated filters badly wrong: a median of 18x off, and up to 90x.
- The correlation-aware estimators fix the estimates and make the optimizer 22 to 30% faster at the same recall.
- Even with perfect estimates, the optimizer picks the best plan only 83% of the time. The hard part is predicting post-filter recall before the query runs, which is what led us to extension B.

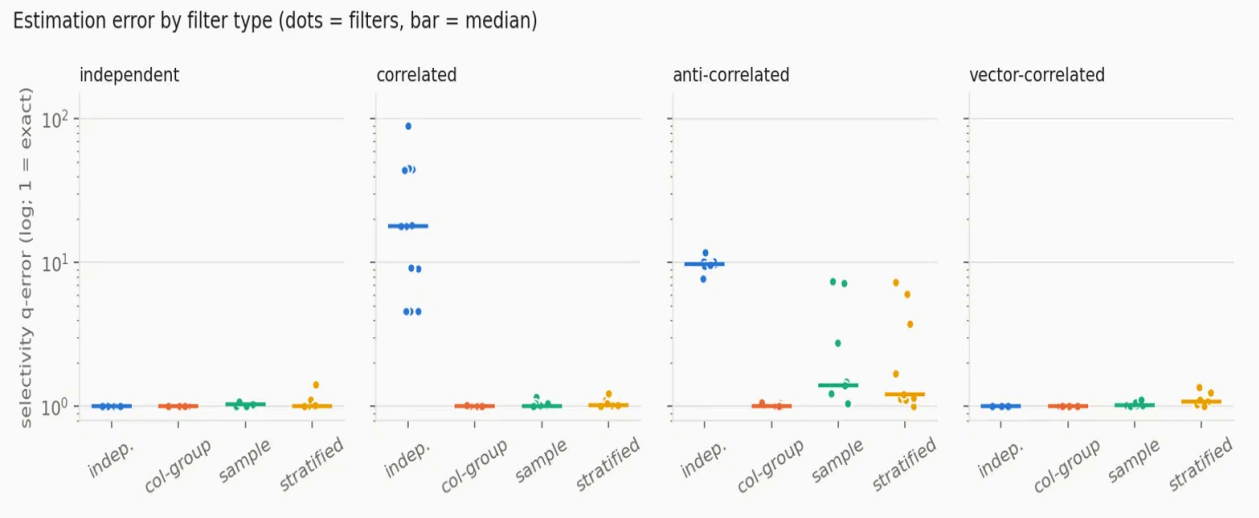


Figure 7. Selectivity estimation error by filter type: the independence assumption is off by up to 90x on correlated filters; the new estimators are close to exact.

5.6 Preliminary results: extension B (synthetic data)

SIFT1M was not available on the machine used for this run, so we built synthetic data that behaves like it: about 95% unfiltered recall while probing 3% of clusters, the same as SIFT1M at nprobe = 32. We used 35 filters with 30 queries each.

Strategy	**Mean latency**	**Queries with recall at least 0.9**
Post-filter only	9.9 ms	6%
Optimizer, default setting (g = 1)	11.1 ms	80%
Optimizer, tuned setting (g = 4)	20.2 ms	97%
Adaptive (ours)	22.4 ms	97%
Pre-filter only	22.5 ms	100%

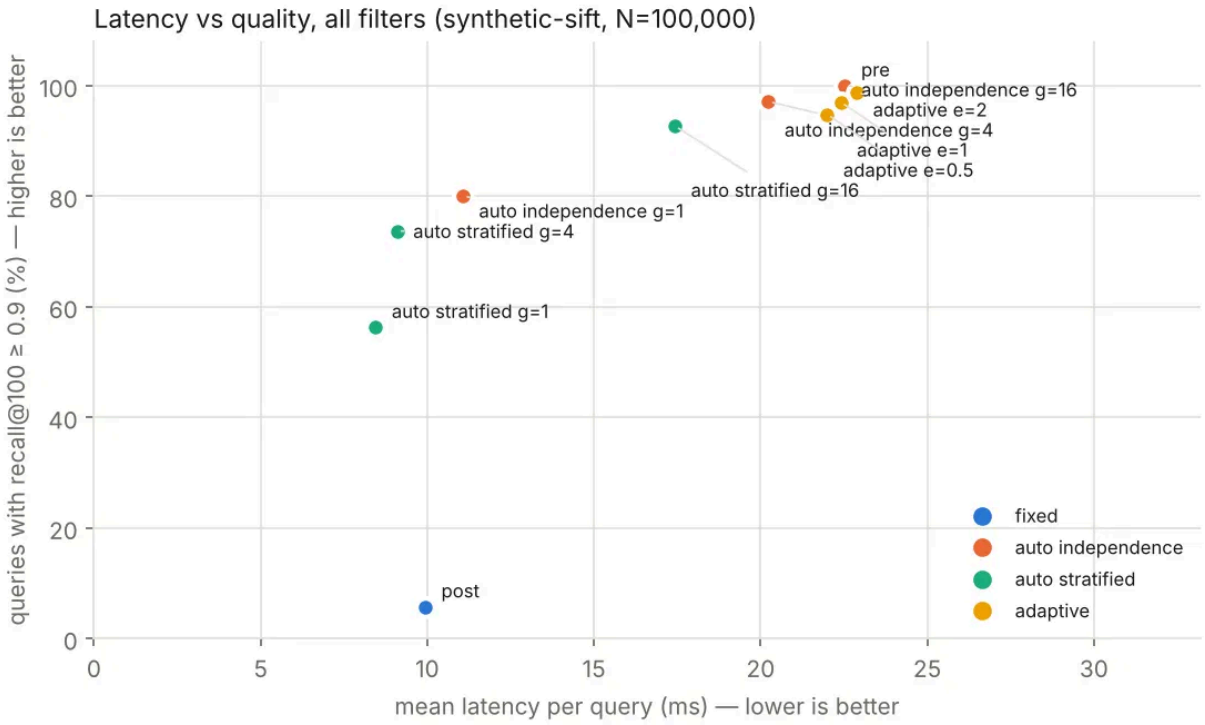


Figure 8. Adaptive search: latency vs share of queries reaching recall 0.9, for each strategy (synthetic, SIFT-like data).

Adaptive search gets the same quality as the tuned optimizer without needing any tuning, but it is about 10% slower. The experiments also caught two bugs in our first versions: stopping as soon as k matches were found gave only 0.41 to 0.49 recall on broad filters, and a cost check that ignored the rest of the scan read too many clusters. We have also run it on GloVe (Section 5.7); a SIFT1M run is still to do.

5.7 Preliminary results: the paper's claims on GloVe

We ran all of the SIFT1M experiments again on GloVe-200 [2], using the ANN-Benchmarks version [3] that matches the paper's Table 2 (1.18M vectors, 200 dimensions). The runs were on Aishwarya's Windows laptop, which is slower than the Mac used for SIFT1M, so we compare the two datasets by recall and ratios, not milliseconds. The paper lists L2 distance for GloVe. We normalised the vectors to unit length, which is how ANN-Benchmarks uses this dataset, so L2 and angular distance rank the neighbours the same way.

#	**Paper claim**	**Our result on GloVe**	**Verdict**
C1	Under 7 ms at 90% recall	Recall 0.67 at nprobe 16; reaching 0.92 takes nprobe 256	Does not hold: GloVe is hard to search
C2	About 10 MB, about 100x less than in-memory	8.7 MB vs 1,575 MB (90 to 181x less)	Holds
C3	Disk speed close to in-memory	About 7x slower, same as SIFT	Does not hold in Python
C4	Bounded build memory, balanced clusters	Build peak 109 MiB; cluster size std 905 down to 16	Holds
C5	Pre-filter exact; plans cross over	Pre-filter recall 1.00; crossover near 0.5%; post-filter recall at most 0.71	Holds
C6	Optimizer picks the right plan	Picks post-filter above 1%, so recall is only 0.36 to 0.71	Partly holds, worse than SIFT
C7	Batching saves about 33%	92% faster per query at batch 1024	Holds, much stronger
C8	Updates stay fast and correct	53 to 199 ms with merges; 1.6 s if never merged	Holds if merged

Why GloVe is harder. Word vectors do not form tight clusters. Without balancing, 262 of the 1,088 clusters hold 10 vectors or fewer, and unfiltered recall at nprobe = 32 is only 0.73 (SIFT: 0.96). Since post-filtering can never beat unfiltered recall, pre-filtering is the only plan that reaches 0.9 recall on filtered queries. Memory, batching and the disk vs memory gap behave just as they did on SIFT.

Extension A on GloVe. The estimation errors repeat SIFT: independence is off by a median of 18x (up to 91x) on correlated filters, and the new estimators bring this close to 1x. But here better estimates did **not** give better plans. The paper's estimator got 60% of queries to recall 0.9, while every corrected estimator, even the exact one, got 52%. This is a negative result, and it supports our SIFT finding that the hard part is predicting post-filter recall, not selectivity.

Extension B on GloVe (35 filters × 30 queries, nprobe = 32):

Strategy	**Mean latency**	**Queries with recall at least 0.9**
Post-filter only	241 ms	7%
Optimizer, default setting (g = 1)	302 ms	60%
Optimizer, tuned setting (g = 16)	838 ms	91%
Adaptive (ours)	1,175 ms	97%
Pre-filter only	1,464 ms	100%

Adaptive search reached 97% of queries on target and was 20% faster than pre-filtering, without any tuning. But because matches near the query were rare, it fell back to pre-filtering on 31 of the 35 filters, so the gain is smaller than we hoped. In short, it helps more when the data clusters well.

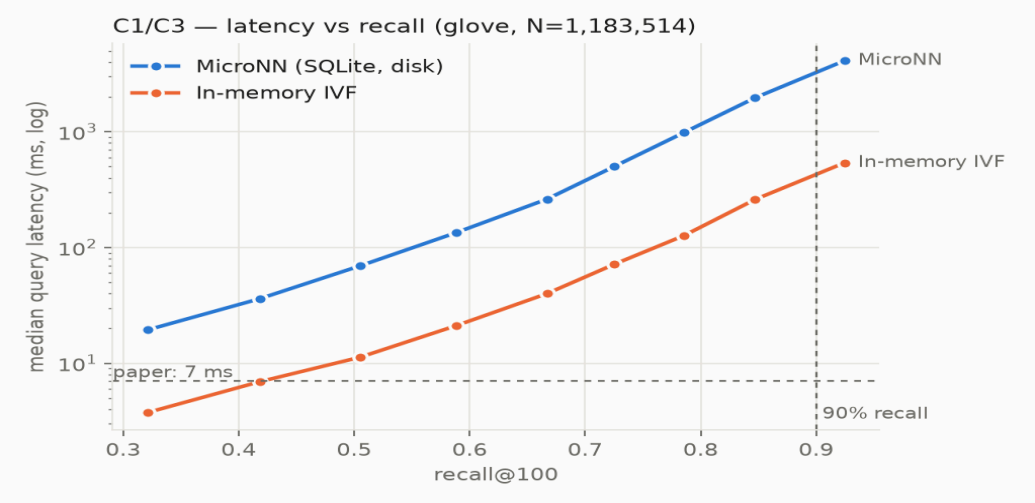


Figure 9. Latency vs recall@100 on GloVe: about 24% of clusters must be read to reach 90% recall.

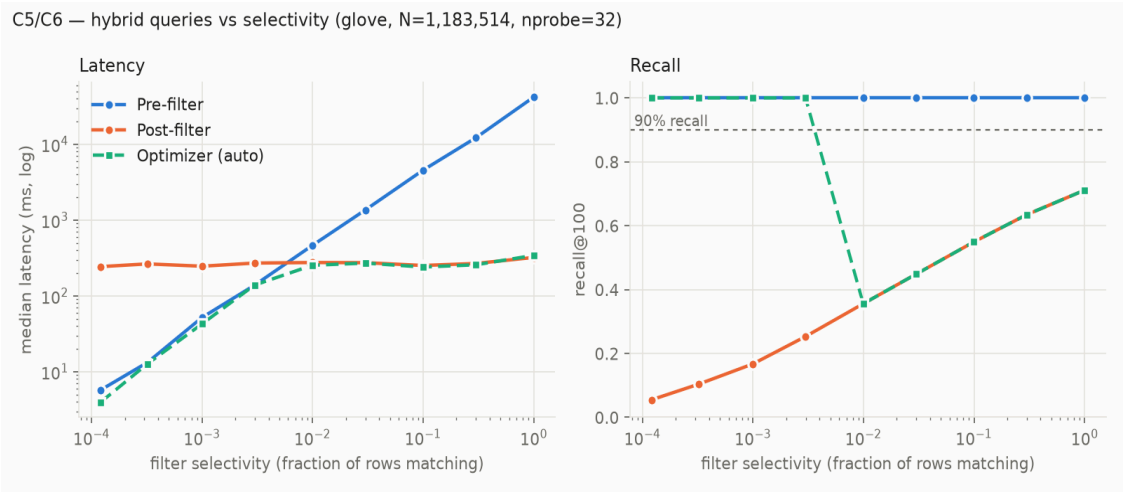


Figure 10. Filtered queries on GloVe: post-filter recall never reaches 0.9, so pre-filter is the only accurate plan.

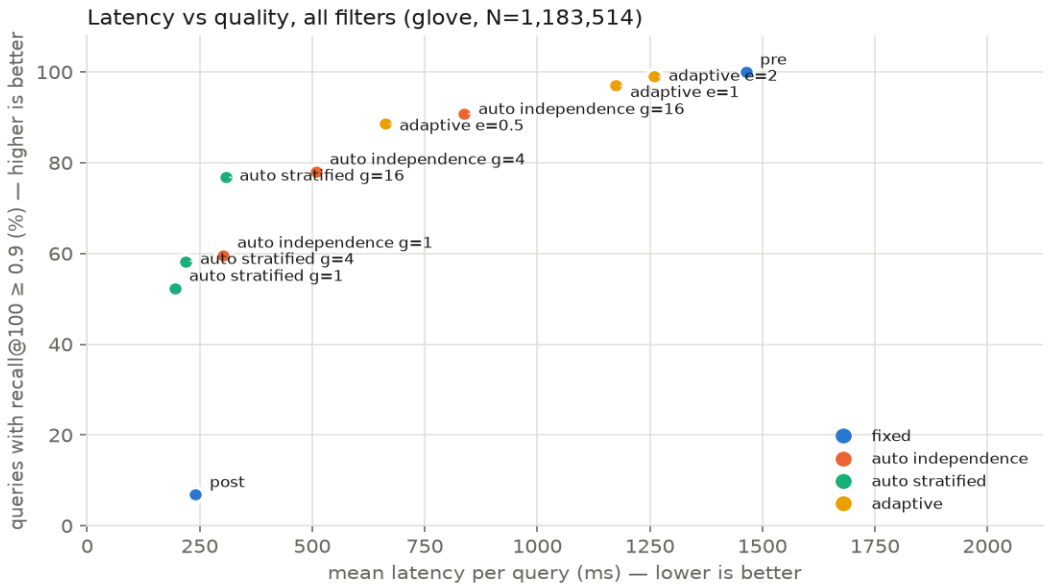


Figure 11. Adaptive search on GloVe: latency vs share of queries reaching recall 0.9, for each strategy.

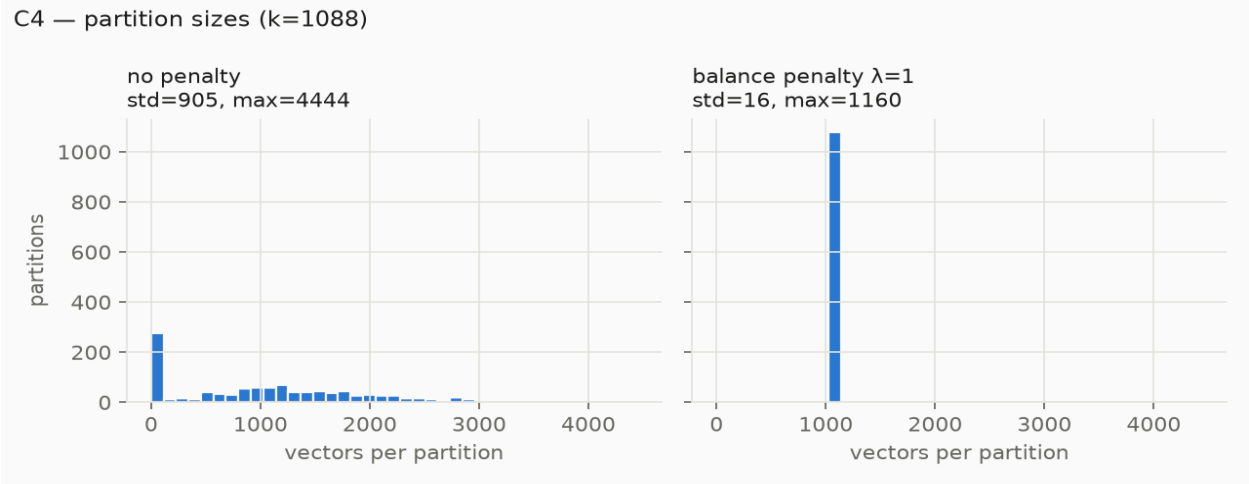


Figure 12. Cluster sizes on GloVe with and without the balance penalty: without it, 262 of the 1,088 clusters are nearly empty.

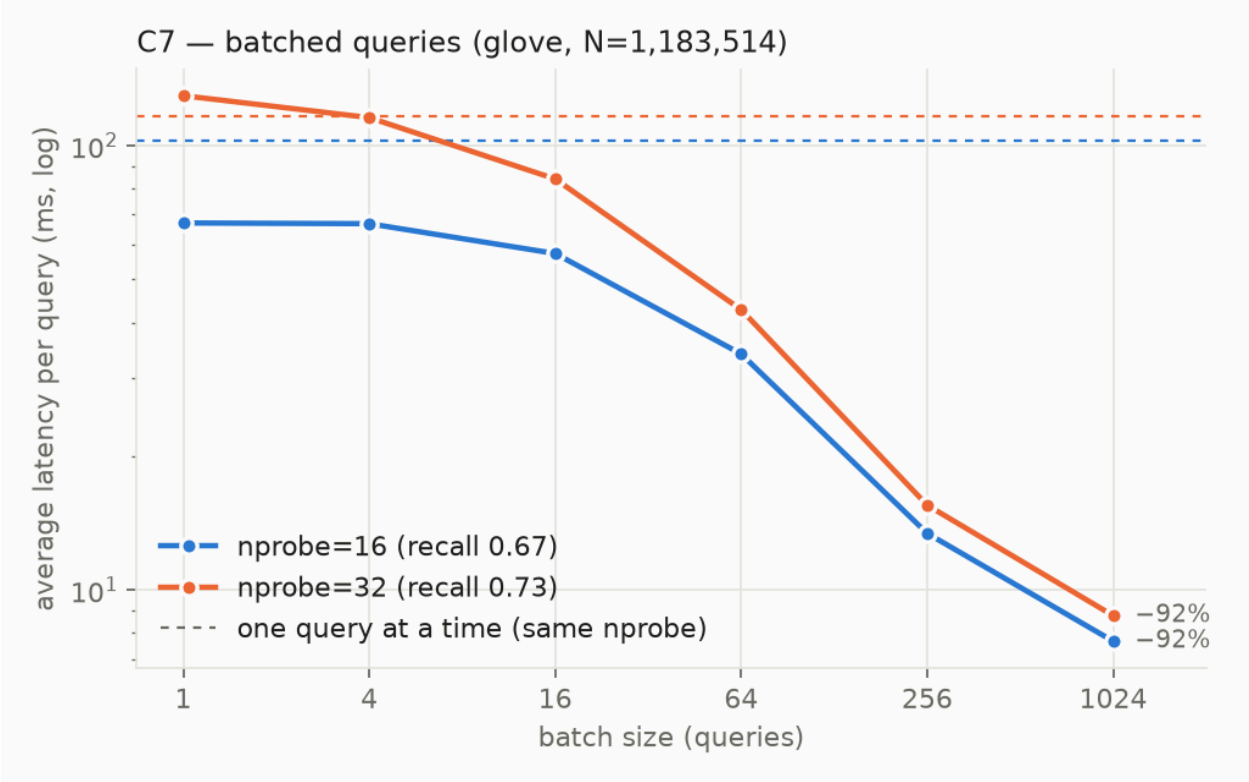


Figure 13. Batching on GloVe: per-query latency drops by 92% at batch size 1024, the same as SIFT.

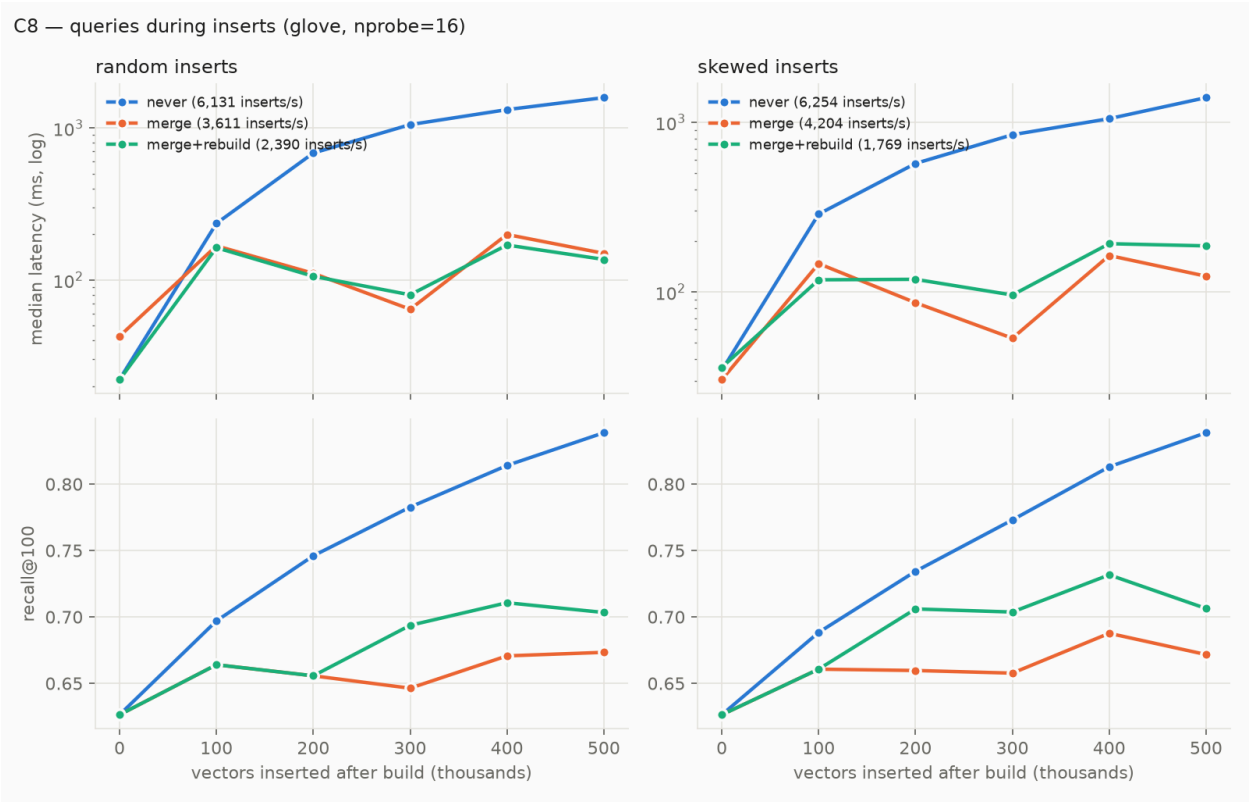


Figure 14. Queries during 500K inserts on GloVe: without merging, latency grows to 1.6 s; merging keeps it between 53 and 199 ms.

5.8 Focused GIST1M experiment status

We implemented the focused GIST1M workflow for MicroNN. The changes add a GIST1M loader with 960-dimensional validation, a basic recall-vs-nprobe sweep, and a self-contained research experiment for estimator q-error and optimizer plan trade-offs. The workflow limits the base index to 100K or 200K vectors so it remains practical in pure Python.

Results so far: the new experiment modules compile successfully, the research-database preparation path passes a smoke test, and the commands are documented in the README. Numerical recall, latency, q-error, and optimizer-trade-off results are not reported yet because the GIST1M files were not available in the development environment; they will be generated by the focused scripts once the dataset is placed under data/gist/.

6. Prototype

The prototype is a Python library with a small API: `create`, `add`, `build`, `analyze`, `search` (with plan = pre, post, auto or adaptive), `delete` and `maintain`. The screenshots below are from our own runs.

Test suite. All 51 tests pass, including the 5 new tests for adaptive search.



Figure 15. Running the test suite.

One filtered search, three plans. We built a 50,000-vector index and ran the same filtered query (one city, before 2015, about 1.2% of rows) with each plan. Pre-filter is exact. Post-filter is fast but only finds 6 of the true top 10, because the matching rows are spread outside the 8 clusters it reads. Adaptive search read 3 clusters, saw that matches were too rare nearby, switched to pre-filter and got all 10.

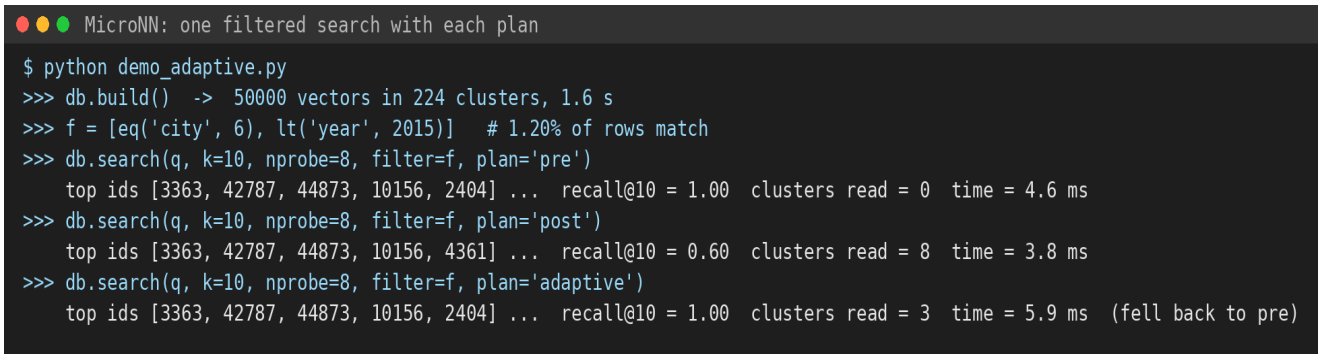


Figure 16. The same query with plan = pre, post and adaptive.

Experiment run. Part of the log from the adaptive search experiment: for each filter it prints the true selectivity, the time and recall of pre-filter, post-filter and adaptive search, how many clusters adaptive read, and how often it fell back to pre-filter.

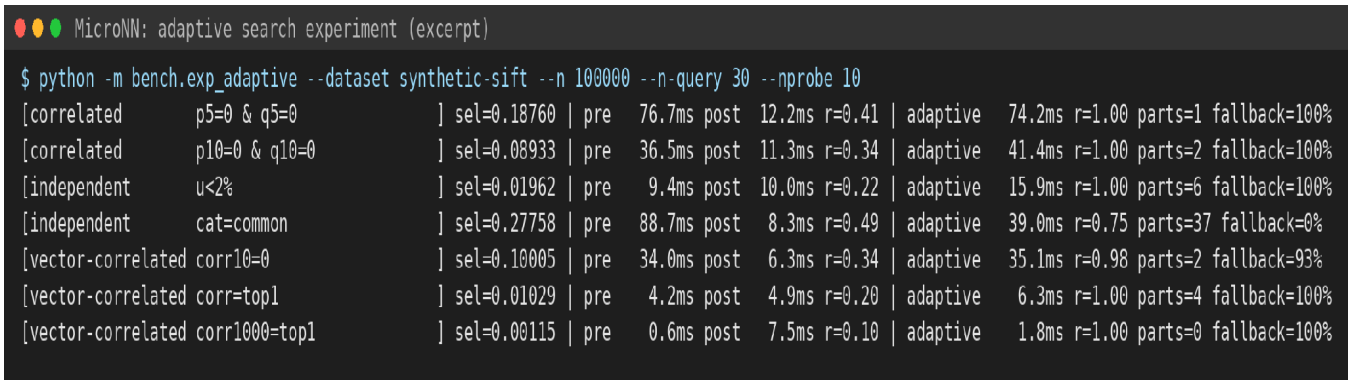


Figure 17. Excerpt of the bench.exp_adaptive log (synthetic, SIFT-like data).

GloVe run on Windows. Part of the batching experiment log on GloVe: per-query time falls from 117 ms one at a time to under 9 ms at batch 1024.

```
[seq ] nprobe=16 102.26 ms/query recall=0.670
[batch] nprobe=16 b=    1 66.96 ms/query (+35% vs seq) recall=0.670 partition scans/query=16.0
[batch] nprobe=16 b=    4 66.70 ms/query (+35% vs seq) recall=0.670 partition scans/query=15.3
[batch] nprobe=16 b=   16 57.15 ms/query (+44% vs seq) recall=0.670 partition scans/query=12.9
[batch] nprobe=16 b=   64 33.96 ms/query (+67% vs seq) recall=0.670 partition scans/query=7.9
[batch] nprobe=16 b=  256 13.42 ms/query (+87% vs seq) recall=0.670 partition scans/query=2.9
[batch] nprobe=16 b= 1024 7.67 ms/query (+92% vs seq) recall=0.670 partition scans/query=1.5
[seq ] nprobe=32 116.62 ms/query recall=0.730
[batch] nprobe=32 b=    1 129.45 ms/query (-11% vs seq) recall=0.730 partition scans/query=32.0
[batch] nprobe=32 b=    4 115.55 ms/query (+1% vs seq) recall=0.730 partition scans/query=28.9
[batch] nprobe=32 b=   16 84.10 ms/query (+28% vs seq) recall=0.730 partition scans/query=20.7
[batch] nprobe=32 b=   64 42.72 ms/query (+63% vs seq) recall=0.730 partition scans/query=9.8
[batch] nprobe=32 b=  256 15.52 ms/query (+87% vs seq) recall=0.730 partition scans/query=3.1
[batch] nprobe=32 b= 1024 8.76 ms/query (+92% vs seq) recall=0.730 partition scans/query=1.6
wrote C:\Users\gnpra\OneDrive\Documents\CS257\MicroNN-main\results\batch_glove_1183514.json
```

Figure 18. bench.exp_batch running on GloVe on a Windows laptop.

7. Next Steps

The first four milestones from our proposal (setup, IVF index, filtered queries with the optimizer, and the cluster-aware estimator) are done, so the project is about halfway. The paper evaluates on SIFT, GloVe and GIST, so we are now running all three. For the second half, each of us will try one new idea on top of the re-implementation and test it on all three datasets, recording both positive and negative results, since knowing what does not work is part of the study. Adaptive search (Section 5.6 and Section 5.7) is the first of these. We have moved the final deliverables up from Dec 10 to Dec 3. The work is in three phases.

Phase	**Task**	**Owner**	**Target**
1. Finish the foundation (Nov 3 to Nov 15)	Run the focused GIST1M recall-vs-nprobe and research experiments on 100K/200K subsets	Anushka	Nov 8
	Run adaptive search on SIFT1M and compare with the synthetic results	Aishwarya , Keerthana	Nov 8
2. Understand and check (Nov 8 to Nov 22)	Check the paper's numbers (7 ms, 10 MB, 33%) against the PDF	All	Nov 8
	Check the selectivity rule against the paper (minimum vs product over conjunctions)	All	Nov 8
	Fix two known issues: measure the cost model after a warm-up, and scale nprobe with the number of clusters in the rebuild test	Keerthana	Nov 12
	Go through the code together and rewrite the main parts in our own words	All	Nov 15
	Each member tries one new idea and tests it on all three datasets	All	Nov 22
3. Final deliverables (Nov 23 to Dec 3)	Final IEEE-format report with UML diagrams	All	Dec 1
	Recorded 15-minute presentation with a live demo	All	Dec 3

Phase	Task	Owner	Target
	README: run instructions and each member's contributions	All	Dec 3

8. Risks and Mitigation

Risk	How we handle it
Python is much slower than the paper's native code, so our times will not match theirs	We compare trends and ratios, and we measured that more than half the gap is the Python SQLite driver
Our filter attributes are synthetic, so results may differ on real metadata	We use several kinds of attributes, and will try a real filtered dataset if time allows
Adaptive search is tested on synthetic data and GloVe, not yet on SIFT1M	Rerun it on SIFT1M before the final report, and label it preliminary until then
GIST (960 dimensions) and GloVe runs take hours on our laptops	GloVe finished in two overnight runs; GIST scripts are ready for 100K and 200K subsets; runs start once the dataset is downloaded.
Uneven workload across the team	Each member owns one module and one report section, tracked through GitHub commits and the README
Busy end of semester (exams and other courses)	Finish the code by Nov 22 and submit everything by Dec 3, a week earlier than our proposal schedule

References

[1] J. Pound et al., "MicroNN: An On-device Disk-resident Updatable Vector Database," in *Companion of the 2025 Int. Conf. on Management of Data (SIGMOD)*, 2025. https://doi.org/10.1145/3722212.3724444

[2] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global Vectors for Word Representation,” in *Proc. EMNLP*, 2014.

[3] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms,” *Information Systems*, vol. 87, 2020.